

Enhancing PIBT via multi-action operations

Egor Yukhnevich¹, **Anton Andreychuk**²

¹HSE University

²AIRI

The League of Robot Runners 2024

1. Path Planning

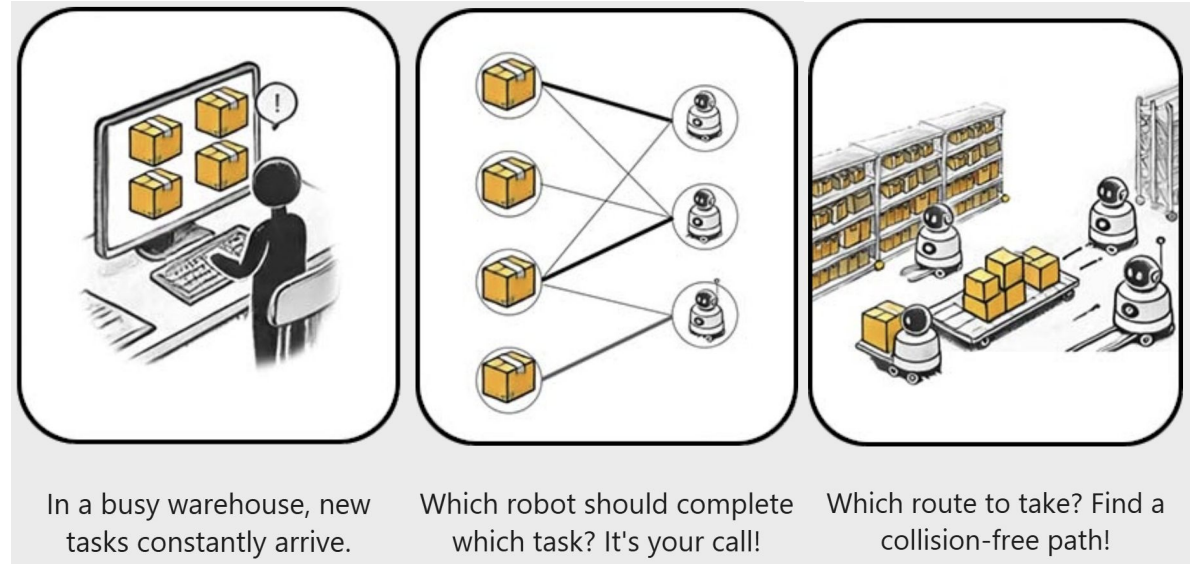
Track

2. Task Scheduling

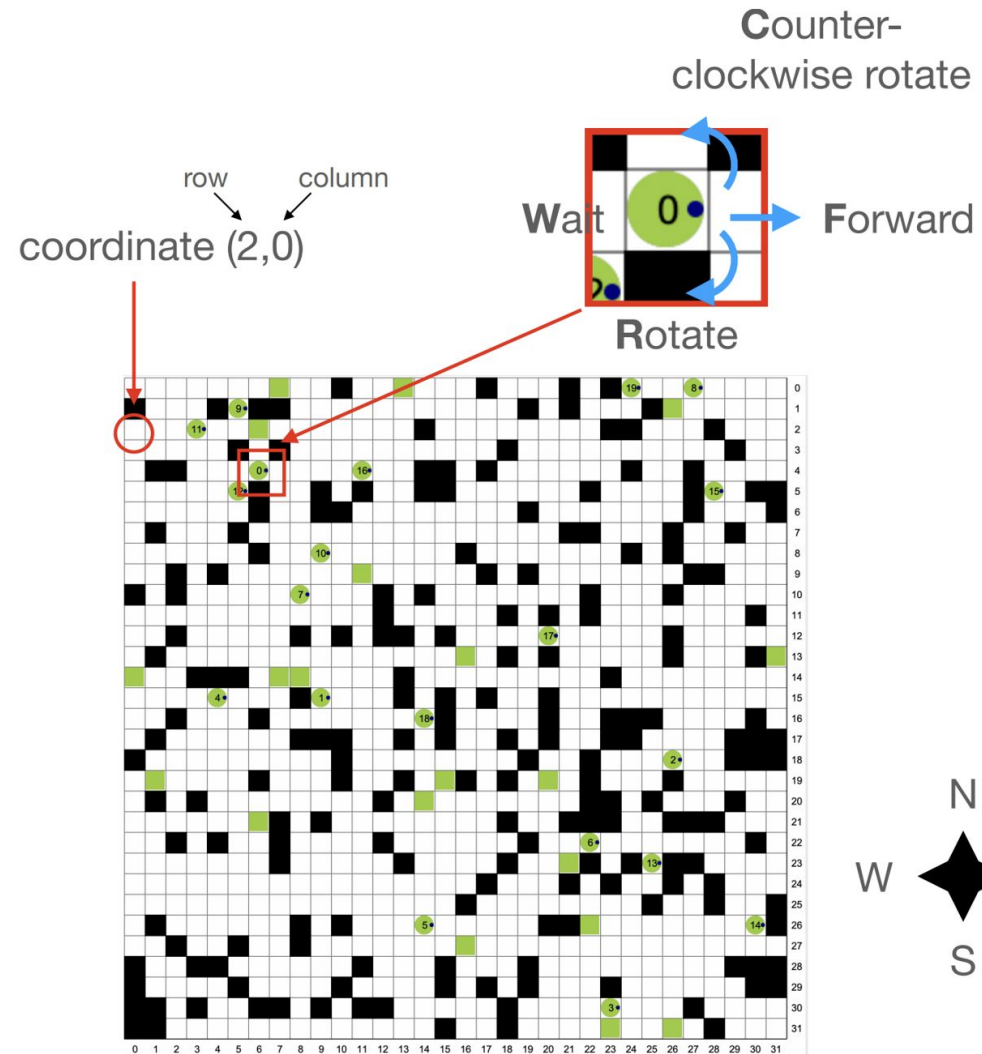
Track

3. Combined Track

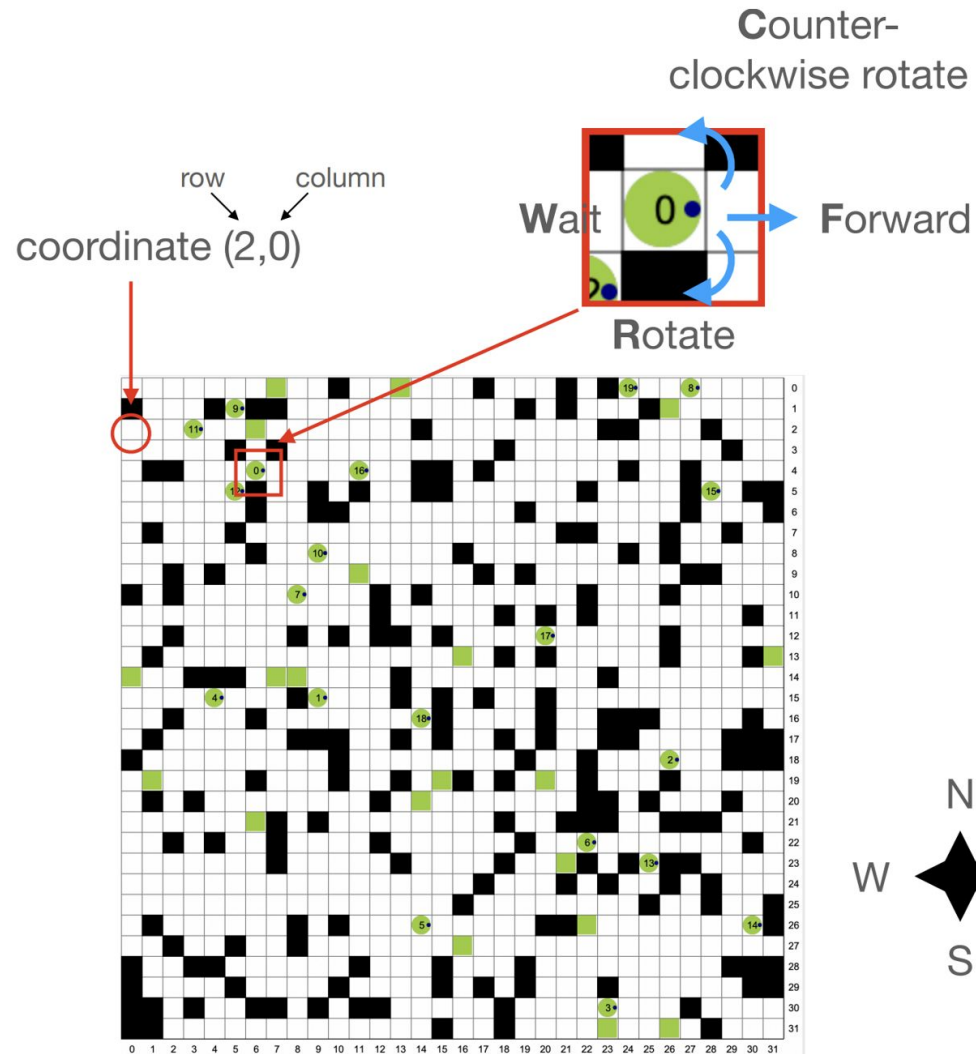
+Line Honours



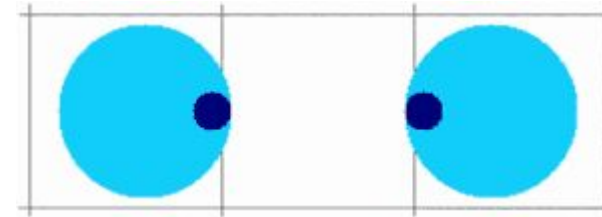
Problem Statement



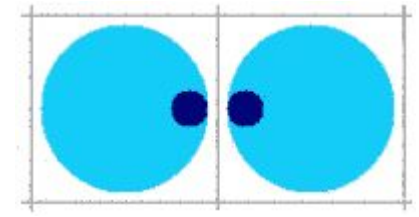
Problem Statement



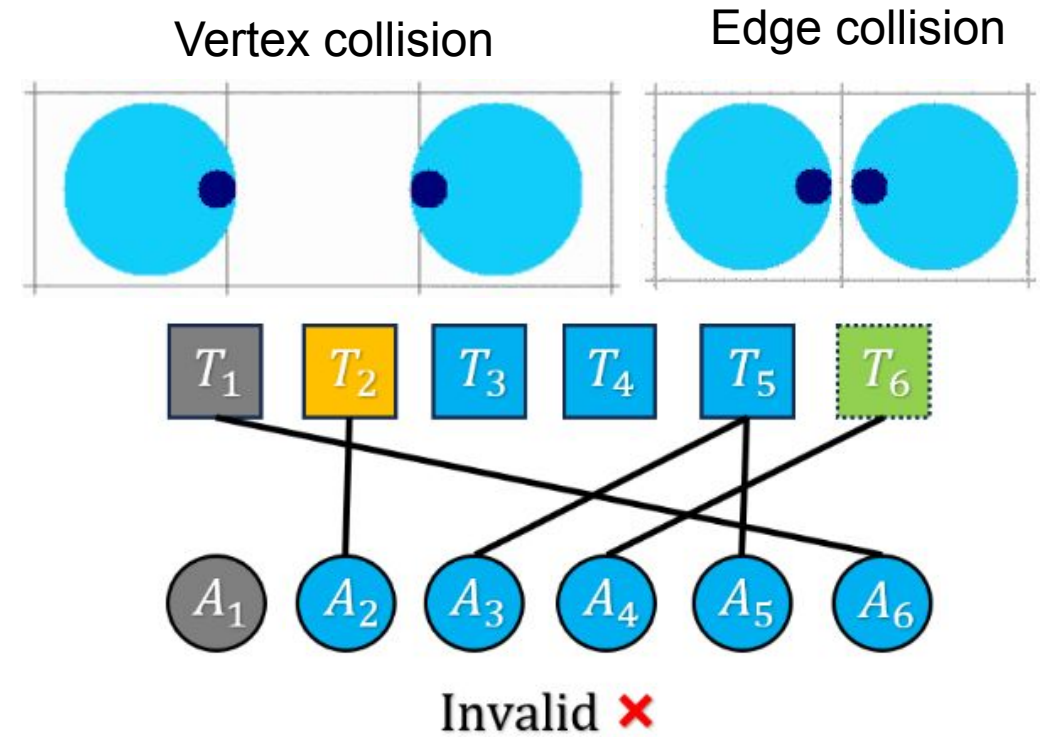
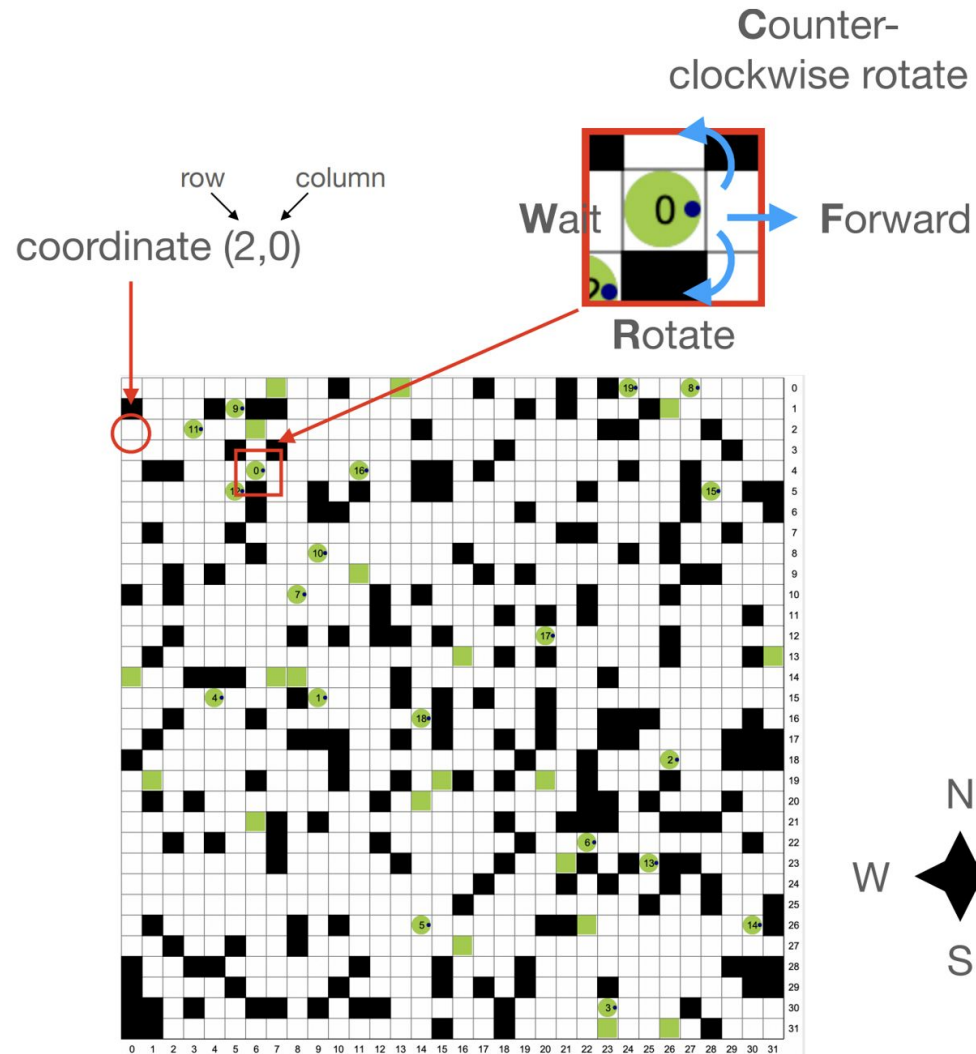
Vertex collision



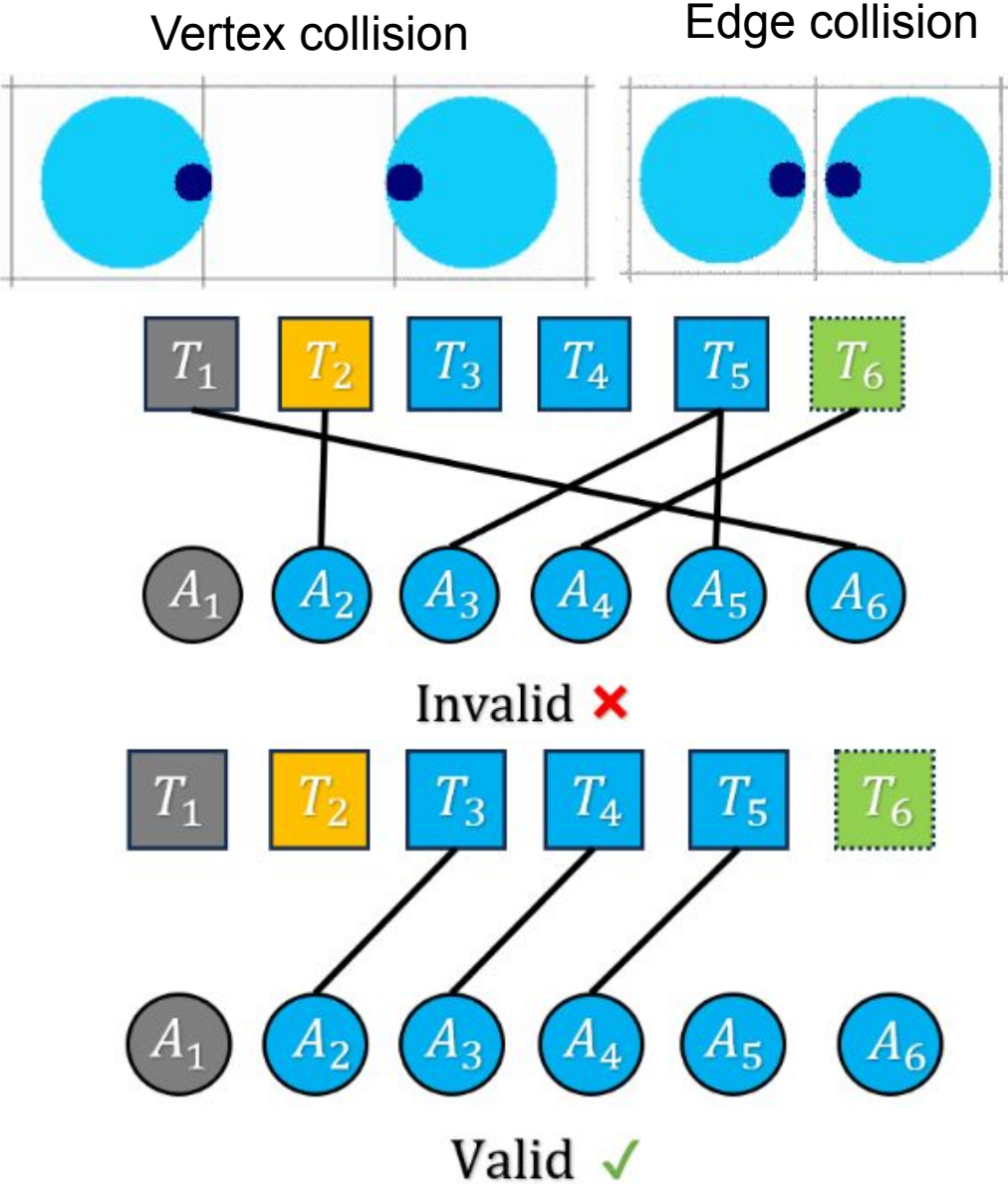
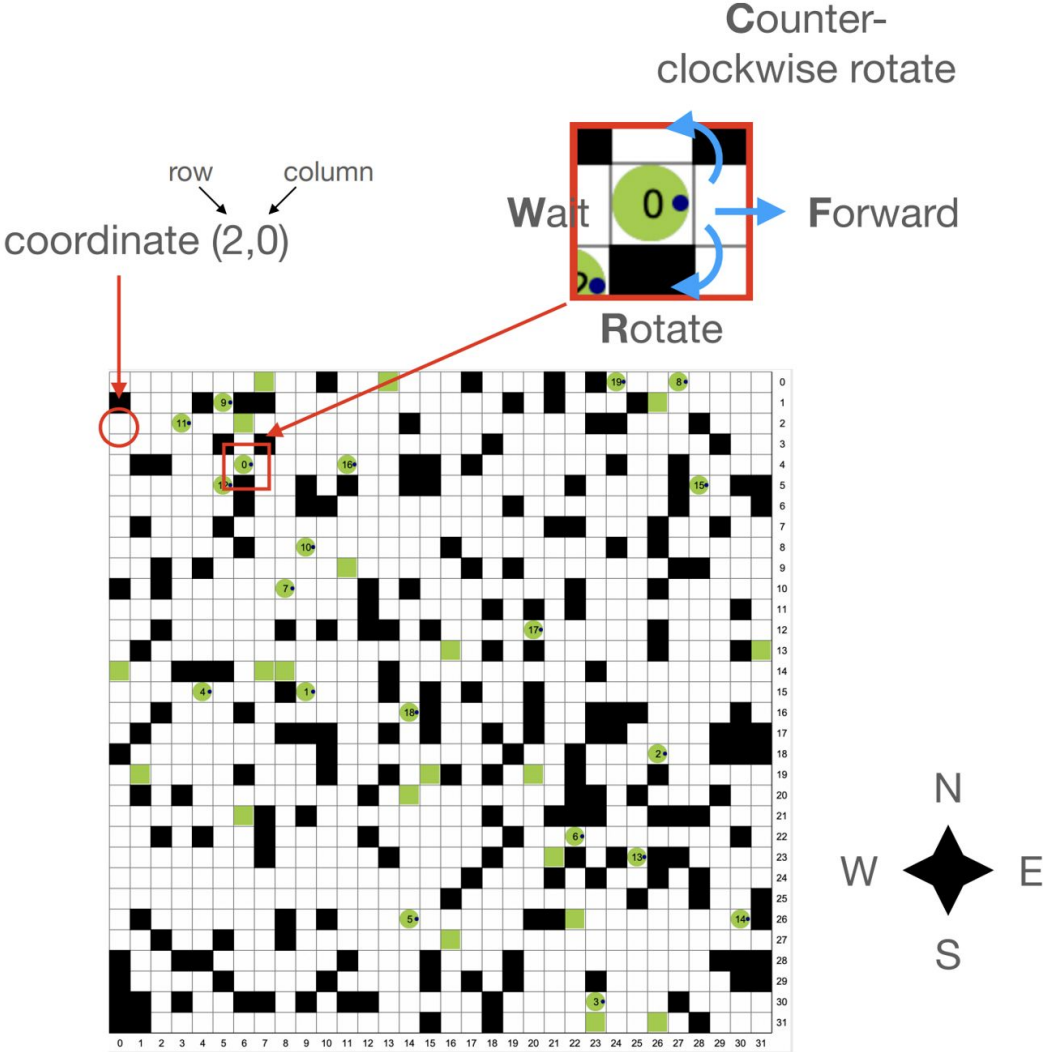
Edge collision

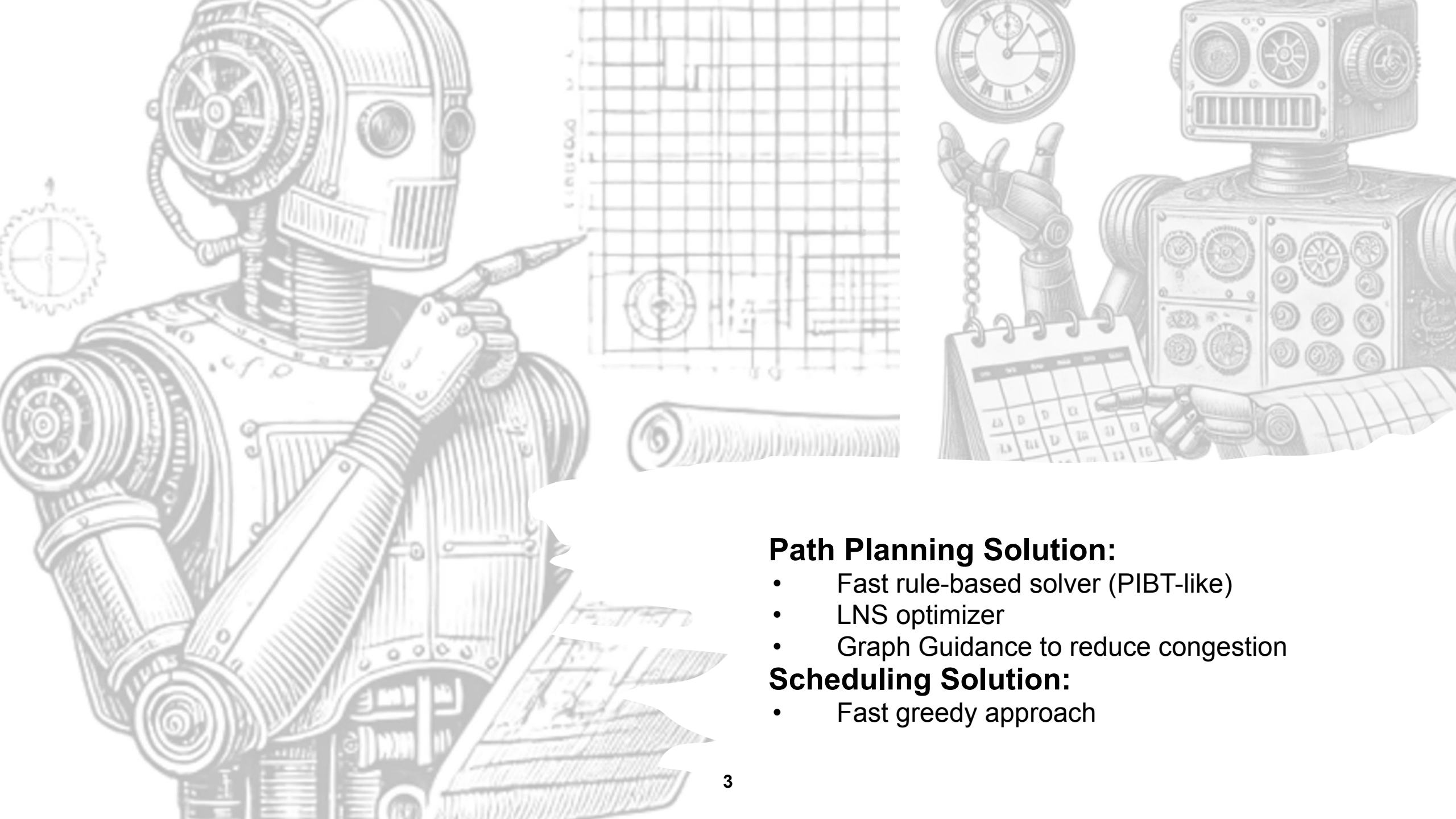


Problem Statement



Problem Statement





Path Planning Solution:

- Fast rule-based solver (PIBT-like)
- LNS optimizer
- Graph Guidance to reduce congestion

Scheduling Solution:

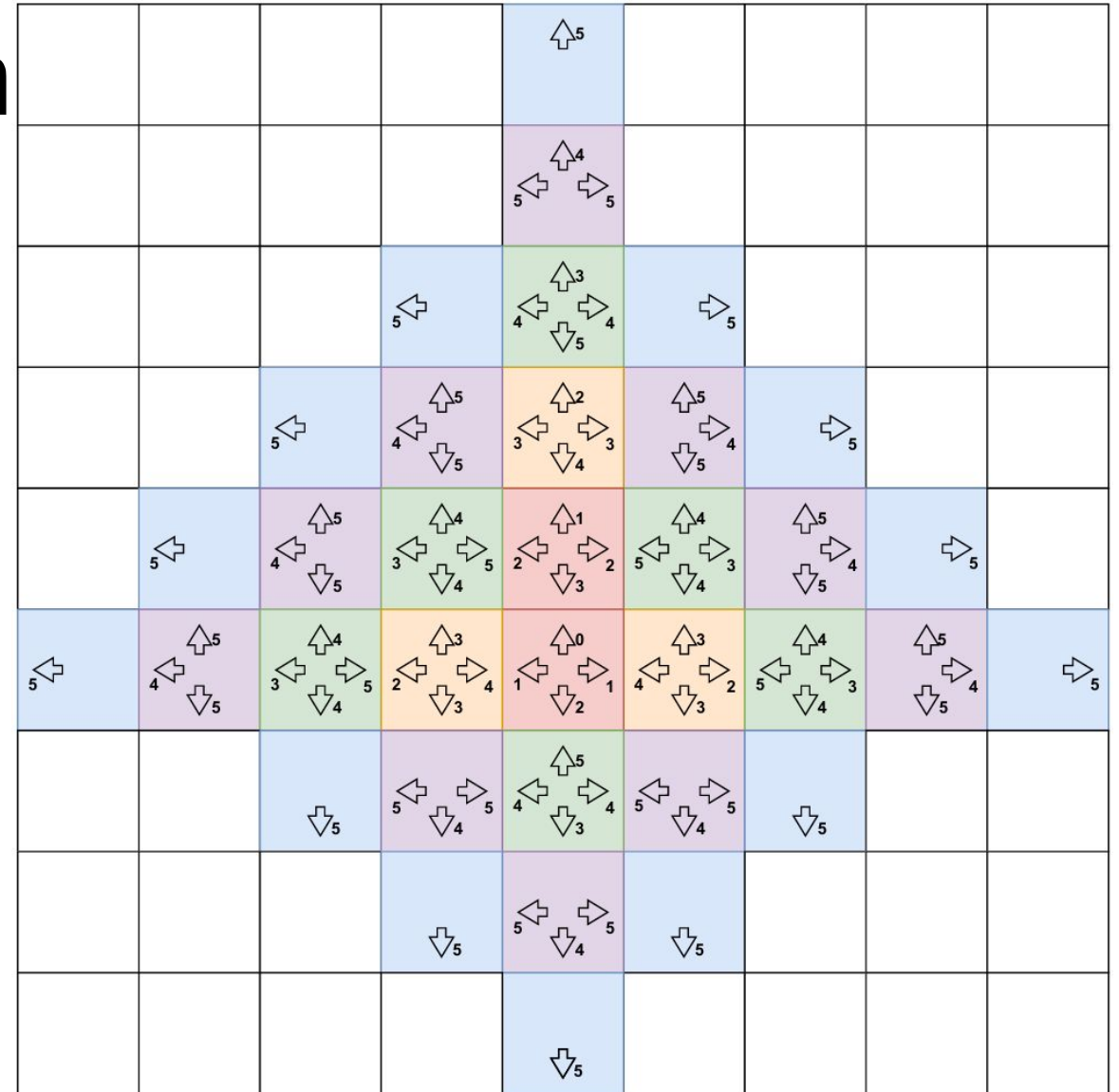
- Fast greedy approach

Concept of operation

- Operation contains of set of actions – FFF, FFW, etc.*
- Rotation action doesn't change the position of the agent => the actual amount of unique cell sequences is significantly less than 4^{length}
- RWF = WRF, CRF=RCF=WWF
- Look forward without planning

Length	1	2	3	4	5
Cells	2	5	11	21	35
States	4	10	23	48	88
Cells sequences	2	6	17	48	136

*R – rotate; F – move forward; W – wait;
C – counter-clockwise rotate;



Enhanced PIBT (EPIBT)

Algorithm 1 EPIBT

```

1: Input: graph  $G$ , starts  $\{s_1, \dots, s_n\}$ , goals  $\{g_1, \dots, g_n\}$ 
2: Output: selected actions  $\{d_1, \dots, d_n\}$ 
3: Preface:  $d_i \leftarrow \{wait\}$  for  $i = 1, \dots, n$ 
4: Preface:  $d'_i \leftarrow \{wait\}$  for  $i = 1, \dots, n$ 
5: Preface:  $P \leftarrow \text{getPath}(s_i, d_i)$  for  $i = 1, \dots, n$ 
6:  $p_i \leftarrow \text{dist}(s_i, g_i)$ ; for each agent  $i = 1, \dots, n$ 
7:  $A \leftarrow \{1, \dots, n\}$ 
8: sort  $A$  in ascending order of priorities  $p_i$ 
9: for  $i \in A$  do
10:   if  $d_i \neq \{wait\}$  then
11:     continue
12:    $P \leftarrow P \setminus \text{getPath}(s_i, d_i)$ 
13:   if  $\text{EPIBT}(i) = \text{failed}$  then
14:      $P \leftarrow P \cup \text{getPath}(s_i, d'_i)$ 

```

```

15: procedure EPIBT( $i$ )
16:    $C \leftarrow op \in \text{Operations}$ 
17:   sort  $C$  in descending order of  $w_{op}$ 
18:   for  $op \in C$  do
19:     if  $\text{getPath}(s_i, op) \not\subset G$  then
20:       continue
21:     if  $\text{getUsed}(s_i, op, P) = \emptyset$  then
22:        $d_i \leftarrow op$ 
23:        $P \leftarrow P \cup \text{getPath}(s_i, op)$ 
24:       return success
25:     if  $|\text{getUsed}(s_i, op, P)| \geq 2$  then
26:       continue
27:      $j \in \text{getUsed}(s_i, op, P)$ 
28:      $P \leftarrow P \setminus \text{getPath}(s_j, d_j)$ 
29:      $P \leftarrow P \cup \text{getPath}(s_i, op)$ 
30:      $d_i \leftarrow op$ 
31:     if  $\text{EPIBT}(j) = \text{success}$  then
32:       return success
33:      $P \leftarrow P \setminus \text{getPath}(s_i, d_i)$ 
34:      $P \leftarrow P \cup \text{getPath}(s_j, d_j)$ 
35:      $d_i \leftarrow d'_i$ 
36:   return failed

```

Enhanced PIBT (EPIBT)

1. Operations are sorted based on w_{op} :

$w_{op} = h(s_i, op, g_i) + \beta_{op}$, where:

β_{op} – some coefficient of the operation for breaking-ties

$h(s_i, op, g_i) = h(s_i, g_i) - h(to(s_i, op), g_i)$,

$h(s, g)$ – distance from s to g ,

$to(s, op)$ – state after performing the op operation

```
15: procedure EPIBT( $i$ )
16:    $C \leftarrow op \in Operations$ 
17:   sort  $C$  in descending order of  $w_{op}$ 
18:   for  $op \in C$  do
19:     if  $getPath(s_i, op) \not\subset G$  then
20:       continue
21:     if  $getUsed(s_i, op, P) = \emptyset$  then
22:        $d_i \leftarrow op$ 
23:        $P \leftarrow P \cup getPath(s_i, op)$ 
24:       return success
25:     if  $|getUsed(s_i, op, P)| \geq 2$  then
26:       continue
27:      $j \in getUsed(s_i, op, P)$ 
28:      $P \leftarrow P \setminus getPath(s_j, d_j)$ 
29:      $P \leftarrow P \cup getPath(s_i, op)$ 
30:      $d_i \leftarrow op$ 
31:     if EPIBT( $j$ ) = success then
32:       return success
33:      $P \leftarrow P \setminus getPath(s_i, d_i)$ 
34:      $P \leftarrow P \cup getPath(s_j, d_j)$ 
35:      $d_i \leftarrow d'_i$ 
36:   return failed
```

Enhanced PIBT (EPIBT)

1. Operations are sorted based on w_{op} :

$w_{op} = h(s_i, op, g_i) + \beta_{op}$, where:

β_{op} – some coefficient of the operation for breaking-ties

$h(s_i, op, g_i) = h(s_i, g_i) - h(to(s_i, op), g_i)$,

$h(s, g)$ – distance from s to g ,

$to(s, op)$ – state after performing the op operation

2. Operations might collide with more than 1 agent – we skip such operations

```
15: procedure EPIBT( $i$ )
16:    $C \leftarrow op \in Operations$ 
17:   sort  $C$  in descending order of  $w_{op}$ 
18:   for  $op \in C$  do
19:     if  $getPath(s_i, op) \not\subset G$  then
20:       continue
21:     if  $getUsed(s_i, op, P) = \emptyset$  then
22:        $d_i \leftarrow op$ 
23:        $P \leftarrow P \cup getPath(s_i, op)$ 
24:       return success
25:     if  $|getUsed(s_i, op, P)| \geq 2$  then
26:       continue
27:      $j \in getUsed(s_i, op, P)$ 
28:      $P \leftarrow P \setminus getPath(s_j, d_j)$ 
29:      $P \leftarrow P \cup getPath(s_i, op)$ 
30:      $d_i \leftarrow op$ 
31:     if EPIBT( $j$ ) = success then
32:       return success
33:      $P \leftarrow P \setminus getPath(s_i, d_i)$ 
34:      $P \leftarrow P \cup getPath(s_j, d_j)$ 
35:      $d_i \leftarrow d'_i$ 
36:   return failed
```


Enhanced PIBT (EPIBT)

1. Operations are sorted based on w_{op} :

$w_{op} = h(s_i, op, g_i) + \beta_{op}$, where:

β_{op} – some coefficient of the operation for breaking-ties

$h(s_i, op, g_i) = h(s_i, g_i) - h(to(s_i, op), g_i)$,

$h(s, g)$ – distance from s to g ,

$to(s, op)$ – state after performing the op operation

2. Operations might collide with more than 1 agent – we skip such operations

3. Priority is used just for ordering – we allow collisions with any agent

```

15: procedure EPIBT( $i$ )
16:    $C \leftarrow op \in Operations$ 
17:   sort  $C$  in descending order of  $w_{op}$ 
18:   for  $op \in C$  do
19:     if  $getPath(s_i, op) \not\subset G$  then
20:       continue
21:     if  $getUsed(s_i, op, P) = \emptyset$  then
22:        $d_i \leftarrow op$ 
23:        $P \leftarrow P \cup getPath(s_i, op)$ 
24:       return success
25:     if  $|getUsed(s_i, op, P)| \geq 2$  then
26:       continue
27:      $j \in getUsed(s_i, op, P)$ 
28:      $P \leftarrow P \setminus getPath(s_j, d_j)$ 
29:      $P \leftarrow P \cup getPath(s_i, op)$ 
30:      $d_i \leftarrow op$ 
31:     if EPIBT( $j$ ) = success then
32:       return success
33:      $P \leftarrow P \setminus getPath(s_i, d_i)$ 
34:      $P \leftarrow P \cup getPath(s_j, d_j)$ 
35:      $d_i \leftarrow d'_i$ 
36:   return failed
  
```

Large Neighborhood Search

- Select random agent r
- Remove path of agent r
- Run EPIBT
 - Any operation with some probability (30%) might be skipped
- If the operations are successfully rebuilt → accept or revert solution using SA (Simulated Annealing)
 - Initial SA temperature = 0.001
 - Every step temperature is multiplied by 0.999

Metric – maximize the reduction of distance to goal for all agents with respect to their priorities

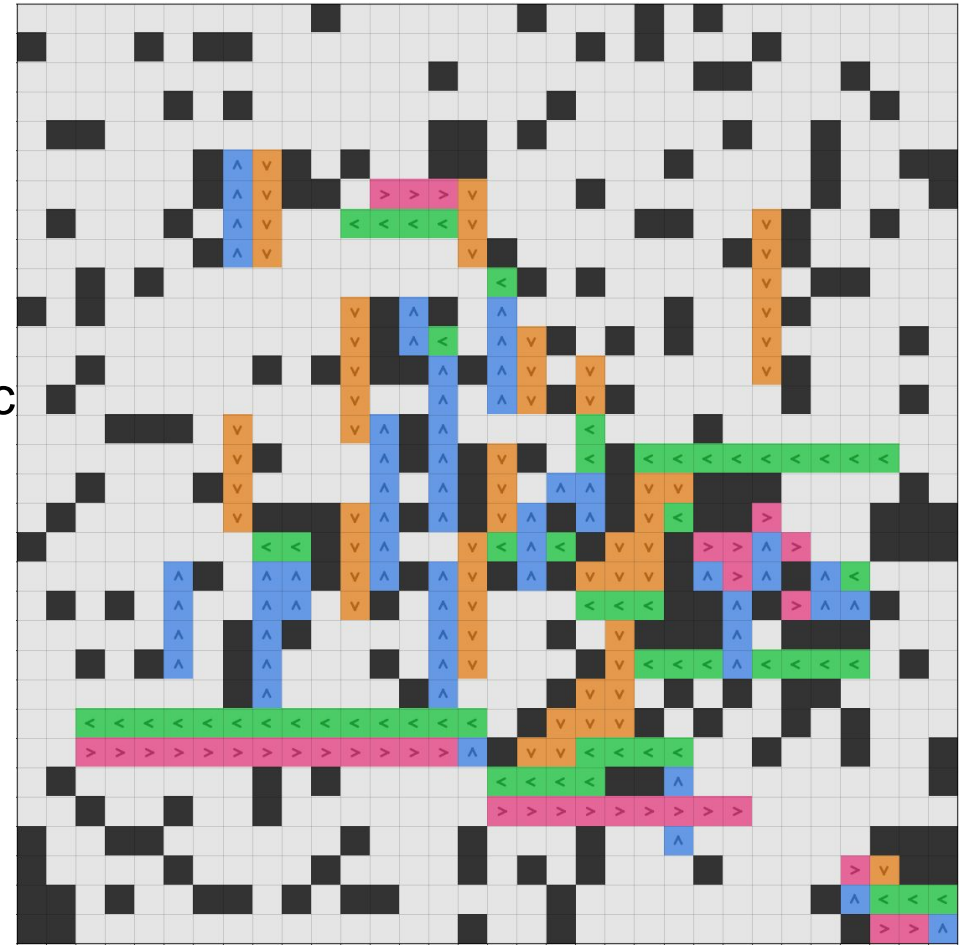
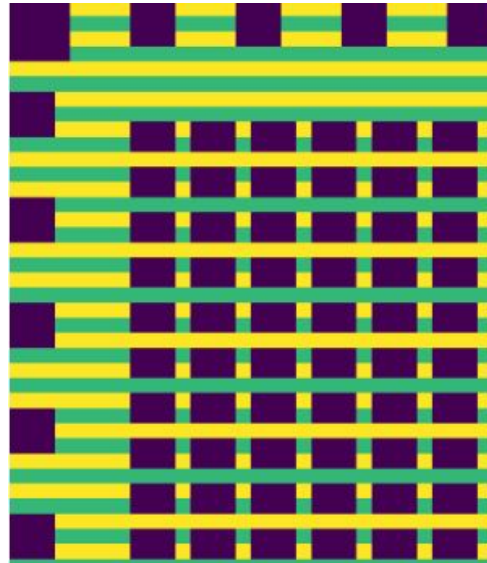
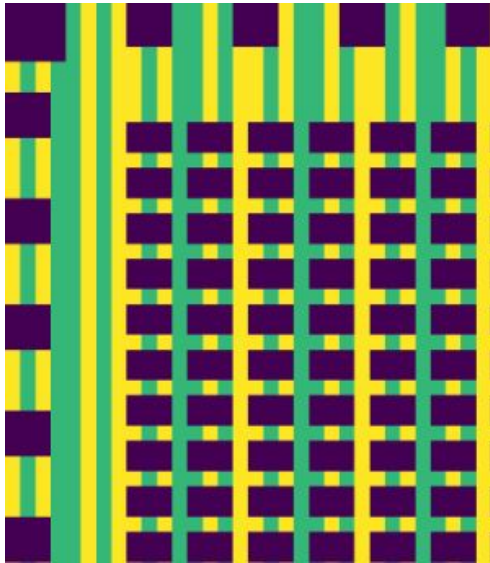
$$\sum_{agent\ r} w_{op} * p_r \rightarrow max$$

Multi-threading utilization:

- Run LNS optimization 32 times
- Choose best found solution
- Version without any intermediate synchronization performed better

Graph Guidance

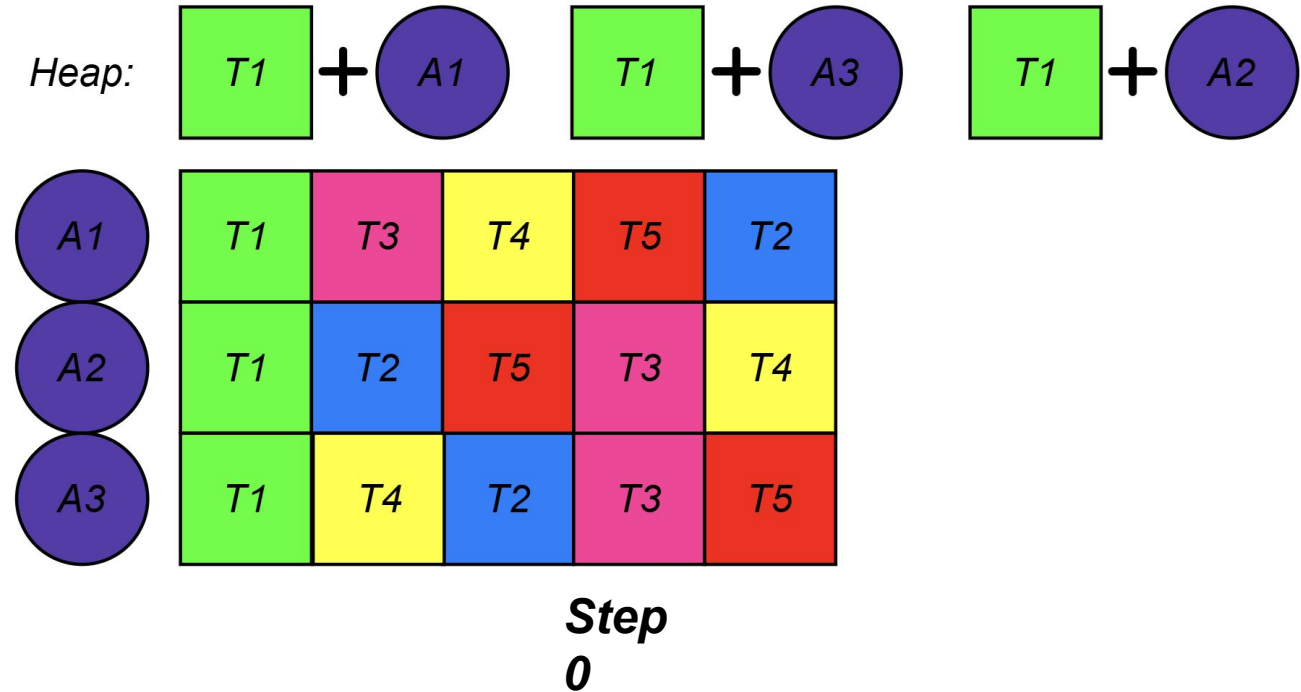
- None of the existing GG approaches was utilized (Traffic Flow, CMA+ES, etc.)
- GG is static, i.e. is not updated during the episode
- Handcrafted for map random-32-32-20
- Rule-based for the rest maps with even-uneven logic
- Penalty for unpreferable actions is 10x cost



Scheduler

Preliminary:

- Compute “cost” of each task – distances between errands
- Compute distances between agents and tasks
- Sort tasks for each agent based on the “full cost”
- Create a heap picking most preferable task for each agent



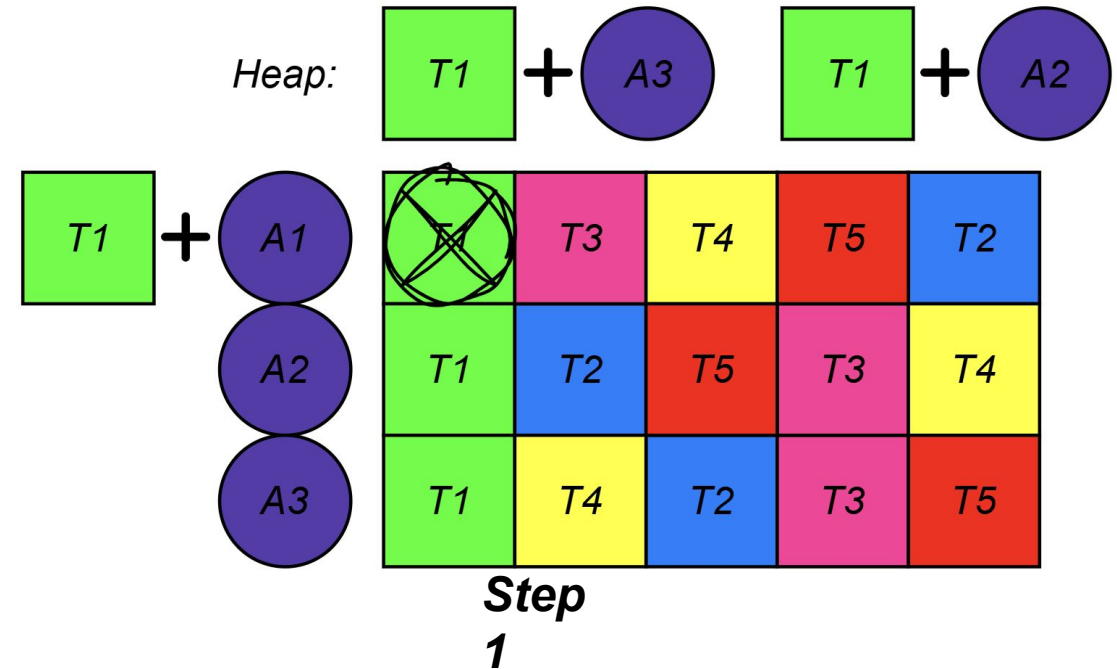
Scheduler

Preliminary:

- Compute “cost” of each task – distances between errands
- Compute distances between agents and tasks
- Sort tasks for each agent based on the “full cost”
- Create a heap picking most preferable task for each agent

Main loop:

- Pick top agent-task pair
- If task is still free – task for this agent is found



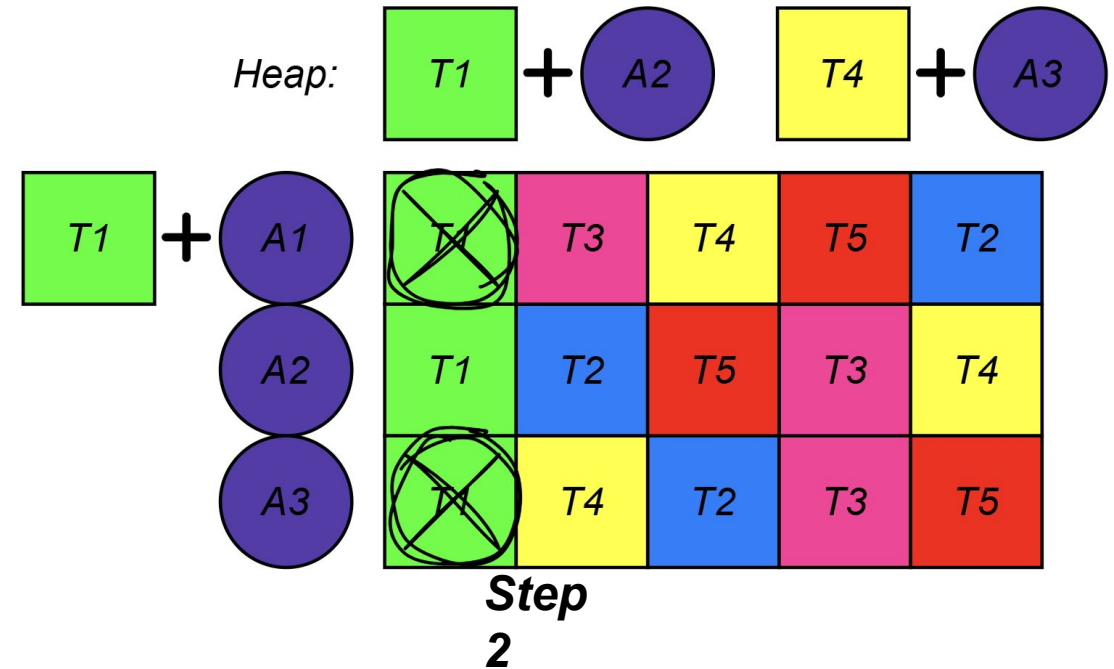
Scheduler

Preliminary:

- Compute “cost” of each task – distances between errands
- Compute distances between agents and tasks
- Sort tasks for each agent based on the “full cost”
- Create a heap picking most preferable task for each agent

Main loop:

- Pick top agent-task pair
- If task is still free – task for this agent is found
- Otherwise put next preferable task into heap



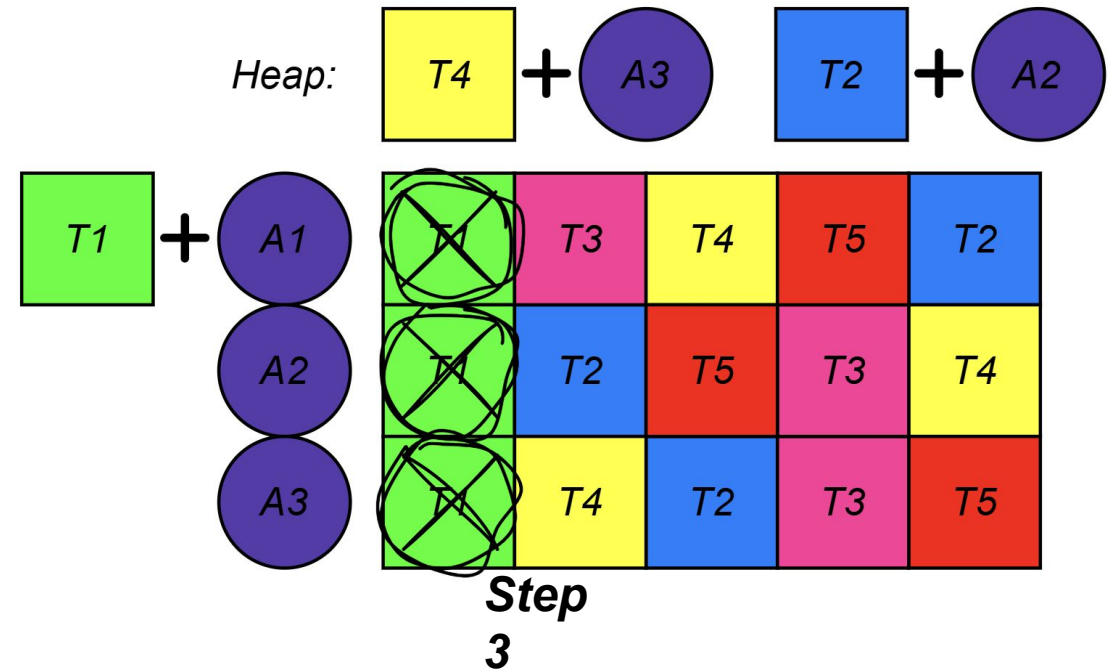
Scheduler

Preliminary:

- Compute “cost” of each task – distances between errands
- Compute distances between agents and tasks
- Sort tasks for each agent based on the “full cost”
- Create a heap picking most preferable task for each agent

Main loop:

- Pick top agent-task pair
- If task is still free – task for this agent is found
- Otherwise put next preferable task into heap
- Continue until all agents have tasks



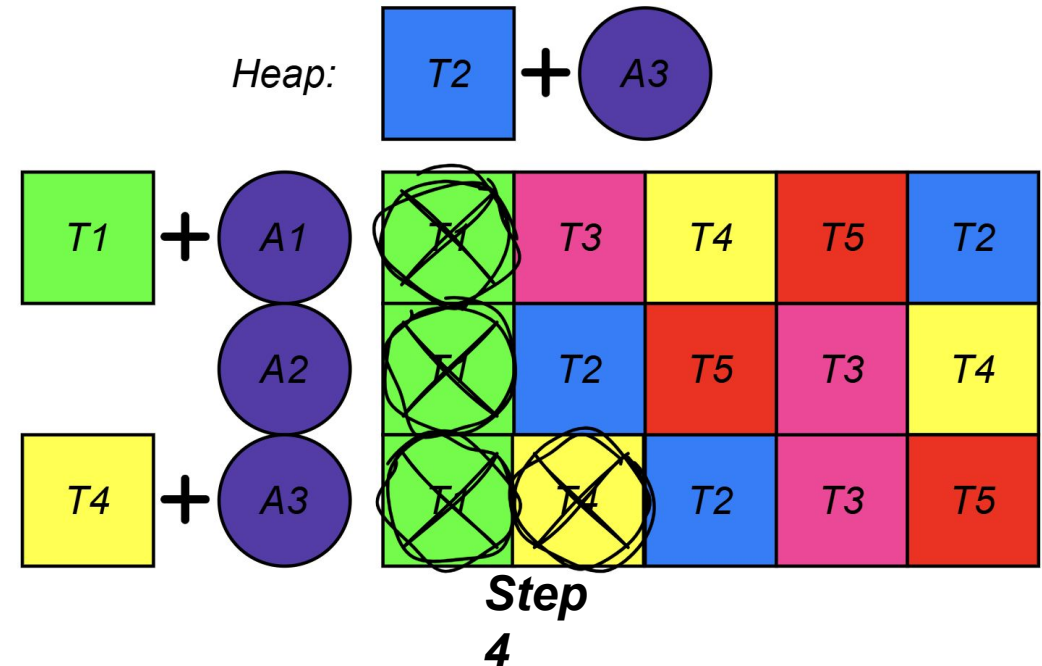
Scheduler

Preliminary:

- Compute “cost” of each task – distances between errands
- Compute distances between agents and tasks
- Sort tasks for each agent based on the “full cost”
- Create a heap picking most preferable task for each agent

Main loop:

- Pick top agent-task pair
- If task is still free – task for this agent is found
- Otherwise put next preferable task into heap
- Continue until all agents have tasks



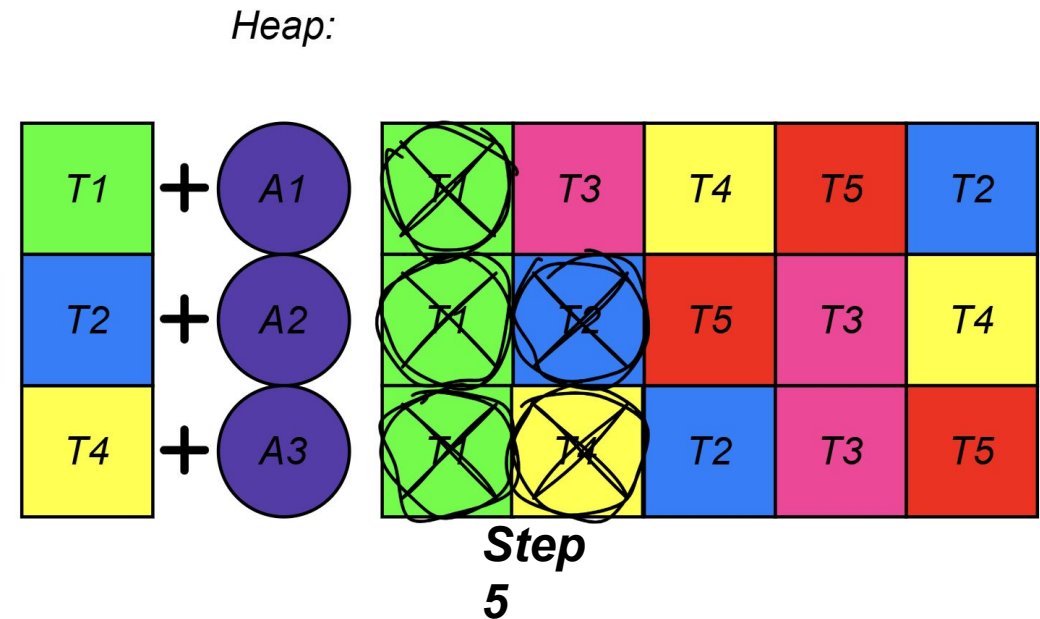
Scheduler

Preliminary:

- Compute “cost” of each task – distances between errands
- Compute distances between agents and tasks
- Sort tasks for each agent based on the “full cost”
- Create a heap picking most preferable task for each agent

Main loop:

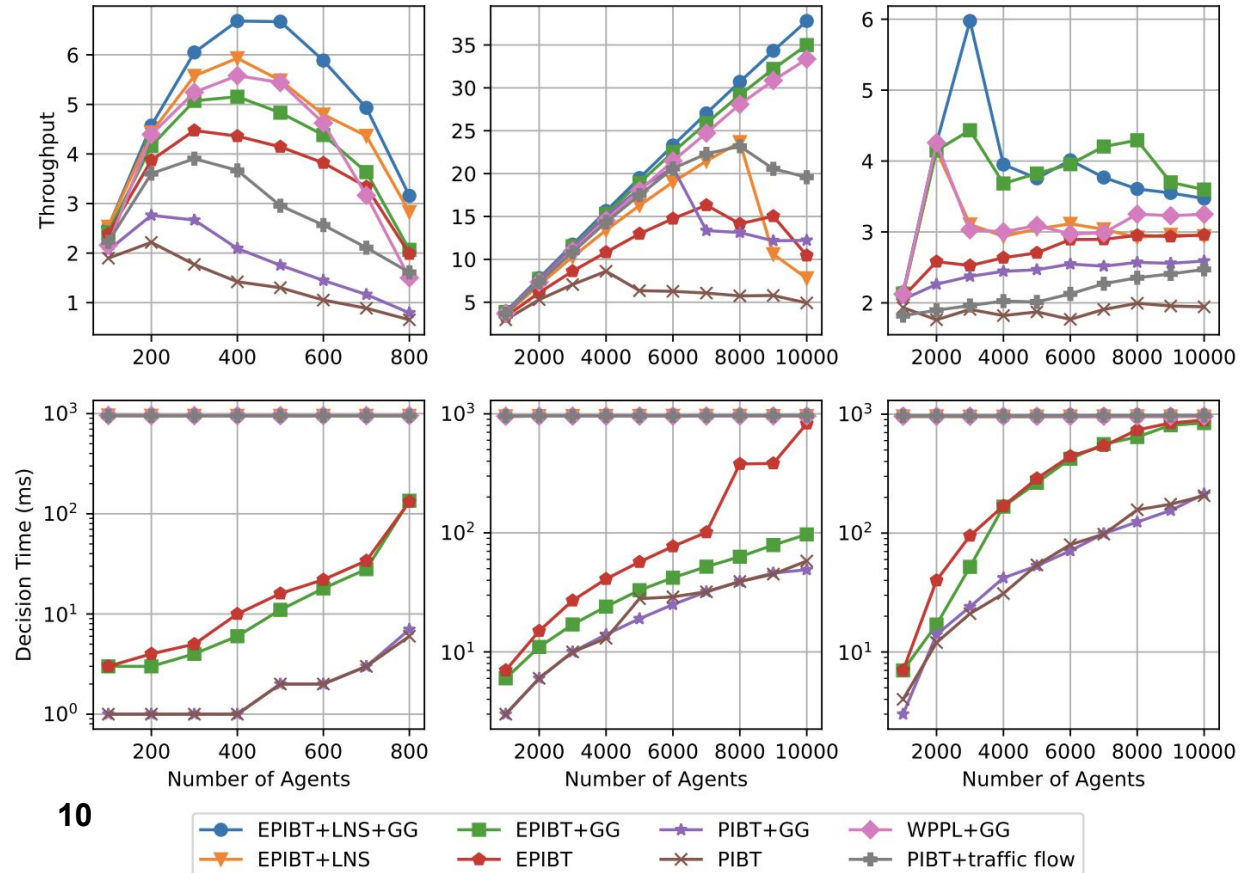
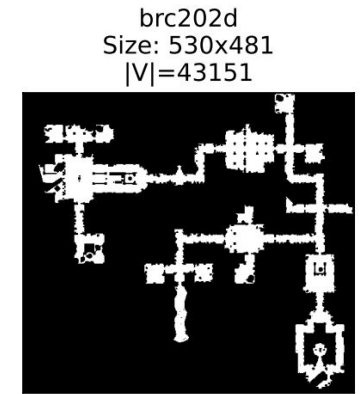
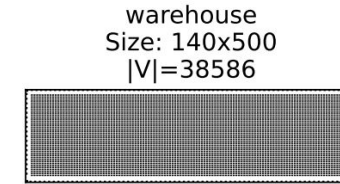
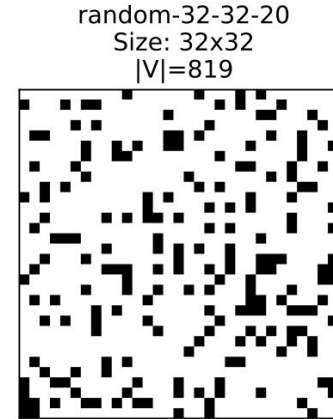
- Pick top agent-task pair
- If task is still free – task for this agent is found
- Otherwise put next preferable task into heap
- Continue until all agents have tasks



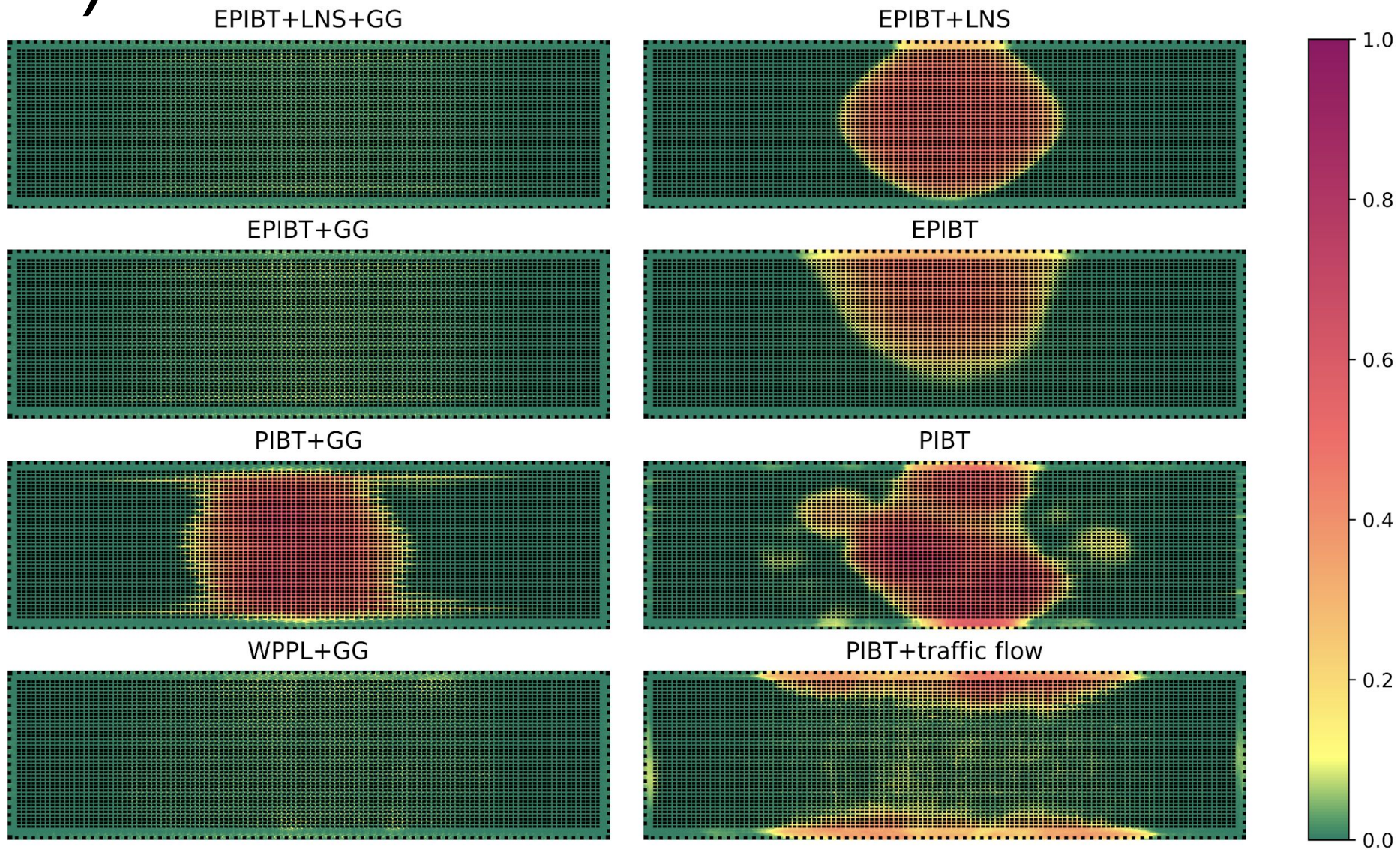
Experimental Evaluatic

Algorithms:

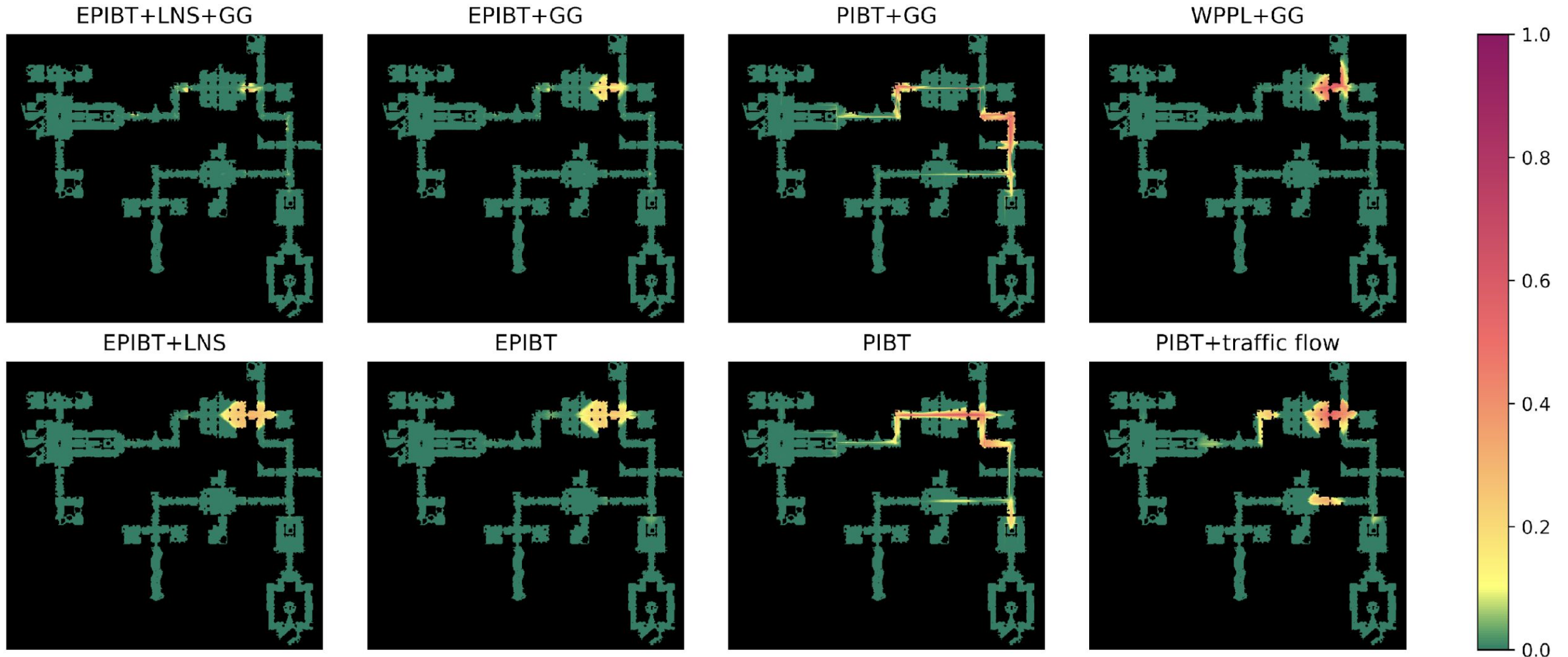
- **PIBT±GG**. Our implementation of the basic algorithm.
- **PIBT+Traffic Flow**. Implementation of the basic planner from the LoRR competition.
- **EPIBT±GG**. Baseline method to estimate how EPIBT performs without LNS.
- **EPIBT+LNS±GG**. The final algorithm.
- **WPPL+GG**. The winner of LoRR 2023.



Waiting Heatmaps – Warehouse (10k agents)



Waiting Heatmaps – brc202d (3k Agents)



Competition and Some Extra Results

Instance	Agents	Steps	Competition Results		PIBT		EPIBT		EPIBT+LNS	
			Tasks	Score	Tasks	Score	Tasks	Score	Tasks	Score
City-01	1500	3000	8420	0.997	7567	0.896	8378	0.992	8369	0.991
City-02	3000	3000	16787	0.987	7912	0.465	15899	0.935	16609	0.976
Game	6500	5000	23274	1	12342	0.530	21690	0.932	23303	1.001
Random-01	100	600	641	0.92	549	0.789	637	0.915	654	0.940
Random-02	200	600	1231	0.977	745	0.591	1066	0.846	1194	0.948
Random-03	400	800	2368	1	746	0.315	1619	0.684	2224	0.940
Random-04	700	1000	2585	1	551	0.213	1630	0.630	2281	0.882
Random-05	800	2000	3050	1	155	0.050	1651	0.541	2669	0.875
Sortation	10000	5000	152714	1	70798	0.464	144616	0.946	148828	0.975
Warehouse	10000	5000	154834	1	51881	0.335	141110	0.911	149526	0.966
Total			365904	9.881	153246	4.648	338296	8.332	355657	9.494

Competition Results

Team Name: Kitty Knight

Team Size: 5

Score Details

Instance	Tasks	Score	Line Honours
CITY-01	8445	1	1
CITY-02	16997	0.999	0
GAME	22604	0.971	0
RANDOM-01	688	0.989	0
RANDOM-02	1260	1	1
RANDOM-03	2084	0.88	0
RANDOM-04	1405	0.544	0
RANDOM-05	2085	0.684	0
SORTATION	150739	0.987	0
WAREHOUSE	152605	0.986	0
Total	358912	9.039	2

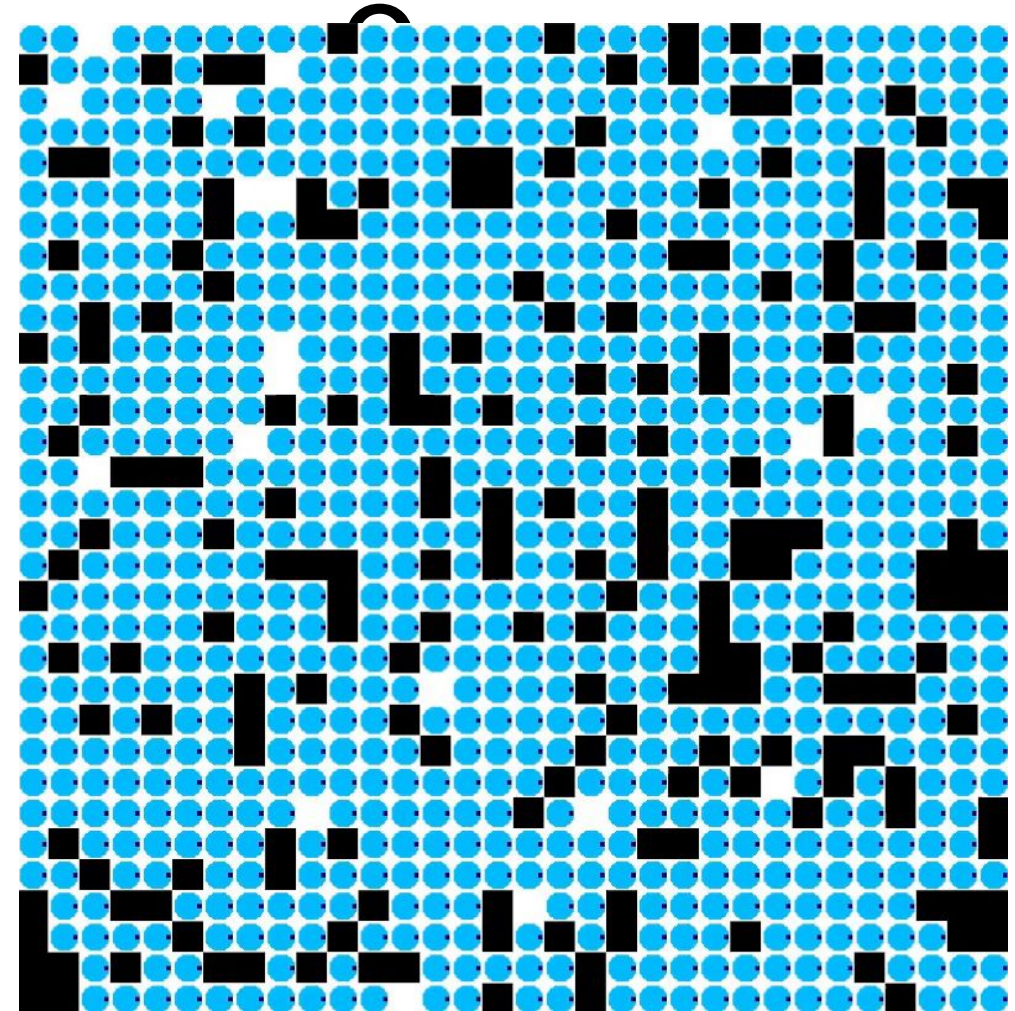
Team Name: No Man's Sky

Team Size: 2

Score Details

Instance	Tasks	Score	Line Honours
CITY-01	8420	0.997	0
CITY-02	16787	0.987	0
GAME	23274	1	1
RANDOM-01	639	0.918	0
RANDOM-02	1221	0.969	0
RANDOM-03	2334	0.986	0
RANDOM-04	2547	0.985	0
RANDOM-05	3050	1	1
SORTATION	152714	1	1
WAREHOUSE	154795	1	0
Total	365781	9.842	3

Dem



Conclusion

	 Combined Track	 Planner Track	 Scheduler Track
 Grand Prize	 Team No Man's Sky	 Team No Man's Sky	 Team No Man's Sky
	 Team Kitty Knight	 Team SYSU-LCIS	 Team verstand
 Distinguished Performance	 Team SYSU-LCIS	 Team RAPID	 Team Zn

- Developing a solver for competition is not the same as doing research:
 - Focus on particular set of instances results in tweaks, tricks and specific optimizations
 - You have less control – cannot modify problem statement, choose baselines, benchmark, etc.
 - Beating existing SOTA approaches is not enough – you need to beat other competitors (that might hide until last day)
 - Simple ideas that work fast and their efficient implementation might surpass a better approach
- There is still a big room for improvement both in terms of the approach and its implementation



github.com/Straple/LORR24